Carbook on Oracle Cloud (Always Free Arm VM) with Coolify

Target: `https://analysight.com` on a `VM.Standard.A1.Flex` (Ubuntu, arm64) running Coolify. Source: `carbook/` in `atifmahmoodai/web-automotive-apps` (branch `main`). The `mobile/` app is not part of this deployment.

1. What Carbook actually is

- **Frontend + backend in one Worker.** It is a Next.js-style app built with **Vinext** (Vite) into a single Cloudflare Worker (`dist/server/index.js`) plus static assets (`dist/client`). The browser calls same-origin routes: `/api/state` , `/api/read` , `/api/action` , `/api/upload` , `/api/media/:id` , `/api/mobile-link` .
- **Cloudflare bindings used:** `DB` = **D1** (SQLite, Drizzle migrations in `drizzle/`), `BUCKET` = **R2** (uploaded photos and videos). No KV, Durable Objects, queues or secrets.
- **Sign-in is the catch.** The app has no login of its own. It trusts `oai-authenticated-user-id` / `-email` headers injected by the ChatGPT "Sites" hosting platform, and links to `/signin-with-chatgpt` . Off that platform, **anyone could send those headers and act as anyone**, so a login proxy in front is required.

2. Recommended approach

Run the built Worker as-is in **workerd** (Cloudflare's open-source runtime, via `wrangler dev --local`). D1 becomes a local SQLite file and R2 becomes local files, both on a Docker volume. Put **oauth2-proxy** (GitHub or Google login) in front of it. A small patch maps the proxy's verified user onto the headers the app already reads.

Option	Verdict
(a) workerd in a container	**Chosen.** No app-logic changes, 1 small auth patch. workerd ships linux-arm64 builds. Tested end to end on the box (pages, migrations, sign-in headers, posting, photo upload and download). Tradeoff: `wrangler dev` is a dev server, which is fine for a small community but not tuned for heavy traffic.
(b) Port to Node + SQLite/disk	Rewrite `cloudflare:workers` env access, D1 `prepare/batch` calls and R2 calls in roughly 6 files, and Vinext has no stable Node target here. Days of work and re-testing. Not worth it now.
(c) Keep the Worker on Cloudflare	Splitting isn't possible because the frontend and backend are one Worker. Hosting it all on Cloudflare means not using the Oracle VM.

3. Code change (one patch)

`self-hosted-auth.patch` changes `worker/index.ts` and `cloudflare-env.d.ts` . It only does something when `SELF_HOSTED=1` , so the existing Sites deployment is unaffected. With that setting:

- it removes any `oai-authenticated-user-*` headers the browser sends,
- it sets them from `X-Forwarded-User` / `X-Forwarded-Email`, which oauth2-proxy adds after login (and strips from client requests),
- `/signin-with-chatgpt` redirects to `return_to`, and `/signout-with-chatgpt` redirects to `/oauth2/sign_out`.

Apply it from the repo root: `git apply --directory=carbook deploy/oracle/self-hosted-auth.patch` (or `patch -p1 < …` from inside `carbook/`).

The "Continue with ChatGPT" button text in `app/components/forms.tsx` is cosmetic, and you can reword it if you like.

4. Files (put them in `carbook/`)

Coolify builds from the repo, so `Dockerfile`, `docker-entrypoint.sh`, `.dockerignore` and `docker-compose.yml` must sit in `carbook/` (copies are in `carbook/deploy/oracle/`).

- `Dockerfile` has two stages on `node:22-bookworm-slim` (glibc is required by workerd, so Alpine won't work). It builds with pnpm 11.25.0 from the lockfile, then the runtime stage gets `wrangler@4.92.0` (with its workerd), plus `dist/` and `drizzle/`.
- `docker-entrypoint.sh` applies D1 migrations, then starts workerd on `:8787` with `--local-upstream $PUBLIC_HOST --upstream-protocol https`. Without these flags the app's same-origin check rejects every POST behind the proxy, which was verified.
- `docker-compose.yml` defines `carbook` (internal only, volume `carbook-data:/data`) and `auth` (oauth2-proxy on `:4180`, the only public service).

5. Oracle VM and Coolify

1. On the VM, open ingress TCP 22/80/443 in the VCN security list and in iptables: `sudo iptables -I INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT && sudo netfilter-persistent save`. Open 8000 only while you set up Coolify.
2. Install Coolify: `curl -fsSL https://cdn.coollabs.io/coolify/install.sh | sudo bash`.
3. In Coolify, create **New Resource → Public/Private Repository** for `atifmahmoodai/web-automotive-apps` with these settings:
   - **Build Pack:** Docker Compose
   - **Branch:** `main`, or the branch that has the patch and files
   - **Base Directory:** `/carbook`
   - **Docker Compose Location:** `/docker-compose.yml`
   - **Domains:** set on the **auth** service only: `https://analysight.com:4180` (add `https://www.analysight.com:4180` too if you want www). Leave `carbook` without a domain.
   - **Persistent storage:** the `carbook-data` volume is created automatically. Back it up, because it holds the database and uploads.
4. **Environment variables** (names only; set values in Coolify):
   - `PUBLIC_HOST`: `analysight.com`
   - `OAUTH2_PROXY_PROVIDER`: `github` (or `google`)

    - `OAUTH2_PROXY_CLIENT_ID`, `OAUTH2_PROXY_CLIENT_SECRET`: from a GitHub OAuth App (or Google OAuth client) with callback URL `https://analysight.com/oauth2/callback`
    - `OAUTH2_PROXY_COOKIE_SECRET`: 32 random bytes, e.g. `openssl rand -base64 32 | tr -- '+/' '-_'`
    - `OAUTH2_PROXY_GITHUB_USERS`: comma-separated GitHub usernames allowed in. If this is empty, any GitHub user can join.
    - `OAUTH2_PROXY_EMAIL_DOMAINS`: keep `*` (or restrict for Google)
5. Deploy. **The first person to sign in becomes the community owner,** so sign in yourself first.

The app expects one origin. If you serve both apex and www, make one of them redirect to the other (for example with a Coolify/Traefik redirect), or the same-origin checks will fail on the non-primary host.

6. DNS for analysight.com

Type	Name	Value
A	`@`	VM reserved public IP
A	`www`	VM reserved public IP (or CNAME `www → analysight.com`)
A	`*` (optional)	VM IP, for future Coolify apps such as `coolify.analysight.com`

If DNS is on Cloudflare, use "DNS only" (grey cloud) until Coolify has issued its Let's Encrypt certificate.

7. Operations

- **Backups:** turn on Oracle boot or block volume backups, and/or regularly copy the `carbook-data` volume (`/data/v3/d1` holds the SQLite database and `/data/v3/r2` holds the media).
- **Updates:** push to the branch and redeploy. Migrations run on every container start, and ones already applied are skipped.
- **Resources:** about 300–500 MB RAM at idle. The Arm VM is plenty.
- **Limits:** this is a single instance with no horizontal scaling. Uploads are capped by the app at 5 MB per photo and 20 MB per video.

8. What was tested (on the build box, 10 Nov 2026)

- A full `docker build` of this Dockerfile (linux/amd64) completed. The arm64 runtime image was built and run under emulation and ran workerd-linux-arm64 correctly. On the Oracle VM the whole build runs natively on arm64. A full arm64 *build* under emulation stalled in the Vite step, which is a QEMU problem, not a code problem.
- Checked in the container: migrations apply, the home page returns 200, spoofed `oai-*` headers are ignored, proxy headers sign the user in, the first user becomes owner, same-origin POSTs work and cross-origin POSTs are rejected, photo upload and download work, data survives a container re-create, and the healthcheck reports healthy.
- **Not tested:** the live oauth2-proxy login against GitHub or Google, and Coolify itself.